DormMate – AI Roommate Compatibility Platform

Project Report

Mentor: Ashok
Issue Date: 4 Jun 2026
Submission Date: 18 Jun 2026

---

1. Project Overview & Objective

The **DormMate – AI Roommate Compatibility Platform** is designed to help students and young professionals find highly compatible roommates based on lifestyle, habits, budgets, and mutual interests. By utilizing automated filtering and weighted algorithmic matching, the platform aims to systematically reduce interpersonal living conflicts and streamline the accommodation discovery process.

2. Problem Statement

Students and young professionals frequently struggle to find compatible roommates when moving to new locations. The absence of a structured matching framework based on vital lifestyle elements—such as sleep schedules, study routines, dietary preferences, and personal cleanliness metrics—often results in unpredictable domestic environments, high roommate turnover rates, and avoidable personal conflicts.

3. System Methodology & Technical Architecture

The platform is engineered using the **Flask** web framework for Python, leveraging semantic frontend rendering via HTML5 and CSS3. In this implementation phase, to bypass the complex overhead of external storage engines during rapid prototyping, the system utilizes an **In-Memory Data Structure (Mock Database)** approach. This state-controlled runtime environment allows fast profile comparison and matrix computations entirely within application memory, establishing a highly reliable Minimum Viable Product (MVP).

4. Lifestyle Questionnaire Design

To accurately capture compatibility indexes, users complete a targeted multi-variable profiling evaluation during user onboarding. The data metrics gathered include:

- **Lifestyle & Sleep Cycles:** Segmenting users into "Morning Person" or "Night Owl" profiles.
- **Cleanliness Standards:** Evaluating self-reported domestic maintenance preferences

("Clean", "Average", "Messy").
- **Hobbies & Social Interests:** Free-text parsing to align interactive preferences (e.g., "coding", "gaming", "music").
- **Financial Filters:** Numerical threshold checks analyzing target accommodation budgets.

5. Compatibility Matching Algorithm Logic

The backend evaluation engine operates on a **Weighted Score Optimization Matrix**. When an active session initiates a request, their parameters are cross-analyzed against the existing record base using programmatic conditional evaluations:

Metric Component	Condition Criterion	Weight Allocation
Lifestyle Alignment	Exact Match (e.g., Morning to Morning)	40% (40 Points Max)
Cleanliness Concurrence	Exact Tier Alignment (e.g., Clean to Clean)	30% (30 Points Max)
Hobby Interconnection	Case-Insensitive String Equivalence Match	30% (30 Points Max)

If parameters diverge, standard baseline index rules (10–20 points) apply to prevent absolute null sets. Matches are sorted dynamically using descending ordering routines (`reverse=True`) before frontend delivery.

6. Well-Documented Source Code

6.1 Application Controller (app.py)

```
from flask import Flask, render_template, request

app = Flask(__name__)

# Mock Database: Multiple dummy users data already stored in app
memory
USERS_DATABASE = [
    {"name": "Sanjay", "age": 20, "hobby": "coding", "lifestyle":
"Morning Person", "cleanliness": "Clean", "budget": 5000},
    {"name": "Karthik", "age": 22, "hobby": "gaming", "lifestyle":
"Night Owl", "cleanliness": "Average", "budget": 4500},
    {"name": "Vijay", "age": 21, "hobby": "music", "lifestyle": "Night
Owl", "cleanliness": "Messy", "budget": 6000},
    {"name": "Arun", "age": 19, "hobby": "coding", "lifestyle":
"Morning Person", "cleanliness": "Clean", "budget": 4800}
]

@app.route("/", methods=["GET", "POST"])
```

```
def home():
    if request.method == "POST":
        current_user = {
            "name": request.form["name"],
            "budget": int(request.form["budget"] or 0),
            "age": request.form["age"],
            "hobby": request.form["hobby"],
            "lifestyle": request.form["lifestyle"],
            "cleanliness": request.form["cleanliness"]
        }

        matches_list = []

        # Comparing current user with all other users in database
        for user in USERS_DATABASE:
            score = 0
            # 1. Lifestyle matching (40 marks)
            if user["lifestyle"] == current_user["lifestyle"]:
                score += 40
            else:
                score += 20

            # 2. Cleanliness matching (30 marks)
            if user["cleanliness"] == current_user["cleanliness"]:
                score += 30
            else:
                score += 10

            # 3. Hobby matching (30 marks)
            if user["hobby"].lower() == current_user["hobby"].lower():
                score += 30
            else:
                score += 10

            matches_list.append({
                "name": user["name"],
                "age": user["age"],
                "hobby": user["hobby"],
                "lifestyle": user["lifestyle"],
                "cleanliness": user["cleanliness"],
                "score": score
            })

        # Sorting matches based on highest score
        matches_list = sorted(matches_list, key=lambda x: x["score"],
reverse=True)

        return render_template("result.html",
```

```
current_name=current_user["name"], matches=matches_list)

    return render_template("index.html")

if __name__ == "__main__":
    app.run(debug=True)
```

6.2 Frontend Intake Layout (index.html)

```
<!DOCTYPE html>
<html>
<head>
    <title>DormMate</title>
    <style>
        body { font-family: Arial; background-color: lightblue;
padding-top: 50px; }
        h1 { color: darkblue; text-shadow: 2px 2px 4px gray;
letter-spacing: 2px; font-size: 36px; }
        p { color: #444; font-size: 18px; }
        input, select { padding: 10px; margin: 8px; width: 250px;
border-radius: 10px; border: 1px solid gray; background-color:
#fafafa; font-size: 15px; }
        button { padding: 12px 25px; font-size: 16px;
background-color: #28a745; color: white; border: none; border-radius:
8px; cursor: pointer; box-shadow: 2px 2px 5px gray; transition: 0.3s;
font-weight: bold; }
        button:hover { opacity: 0.8; background-color: #218838;
transform: scale(1.1); }
        form { background-color: white; padding: 20px; display:
inline-block; border-radius: 10px; box-shadow: 0px 4px 15px
rgba(0,0,0,0.2); max-width: 400px; margin-top: 20px; }
    </style>
</head>
<body>
<center>
    <h1>DormMate - AI Roommate Matching</h1>
    <p>Find your perfect roommate based on your preferences</p>
    <form method="POST">
        <input type="text" name="name" placeholder="Enter Name"
required><br><br>
        <input type="number" name="budget" placeholder="Enter Budget"
required><br><br>
        <input type="number" name="age" placeholder="Enter Age"
required><br><br>
        <input type="text" name="hobby" placeholder="Enter Hobby"
required><br><br>
        <select name="lifestyle">
```

```
            <option>Morning Person</option>
            <option>Night Owl</option>
        </select><br><br>
        <select name="cleanliness">
            <option>Clean</option>
            <option>Average</option>
            <option>Messy</option>
        </select><br><br>
        <button type="submit">Submit</button>
    </form>
</center>
</body>
</html>
```

6.3 Dynamic Matrix Template (result.html)

```
<!DOCTYPE html>
<html>
<head>
    <title>Match Results</title>
    <style>
        body { font-family: Arial, sans-serif; background-color:
#f5f5f5; text-align: center; padding: 20px; }
        .container { max-width: 600px; margin: auto; background:
white; padding: 20px; border-radius: 10px; box-shadow: 0px 4px 10px
rgba(0,0,0,0.1); }
        .match-card { background-color: #eef2f7; border-left: 5px
solid #28a745; margin: 15px 0; padding: 15px; border-radius: 5px;
text-align: left; }
        .score { float: right; color: #d9534f; font-weight: bold;
font-size: 20px; }
    </style>
</head>
<body>
<div class="container">
    <h1>Roommate Matching Results</h1>
    <h2>Welcome, {{ current_name }}!</h2>
    <p>Here are the best compatible roommates found for you:</p>
    <hr>
    {% for match in matches %}
    <div class="match-card">
        <span class="score">{{ match.score }}% Match</span>
        <h3>Name: {{ match.name }}</h3>
        <p><strong>Age:</strong> {{ match.age }} |
<strong>Hobby:</strong> {{ match.hobby }}</p>
        <p><strong>Lifestyle:</strong> {{ match.lifestyle }} |
<strong>Cleanliness:</strong> {{ match.cleanliness }}</p>
```

```
    </div>
    {% endfor %}
    <br>
    <a href="/" style="text-decoration: none; color: #007bff;
font-weight: bold;">← Find Again</a>
</div>
</body>
</html>
```

7. Future Enhancements & Scalability

While the present architecture excels in performance speeds via In-Memory data parsing, prospective production roadmaps include migrating state-level elements into external persistent storage networks (such as **Supabase** or **Firebase Studio**) to host concurrent users. Furthermore, standard weighted parameters can seamlessly transition into deep-learning similarity matching mechanisms via the Gemini API to execute automated lifestyle agreements and advanced multi-tenant cluster identification.